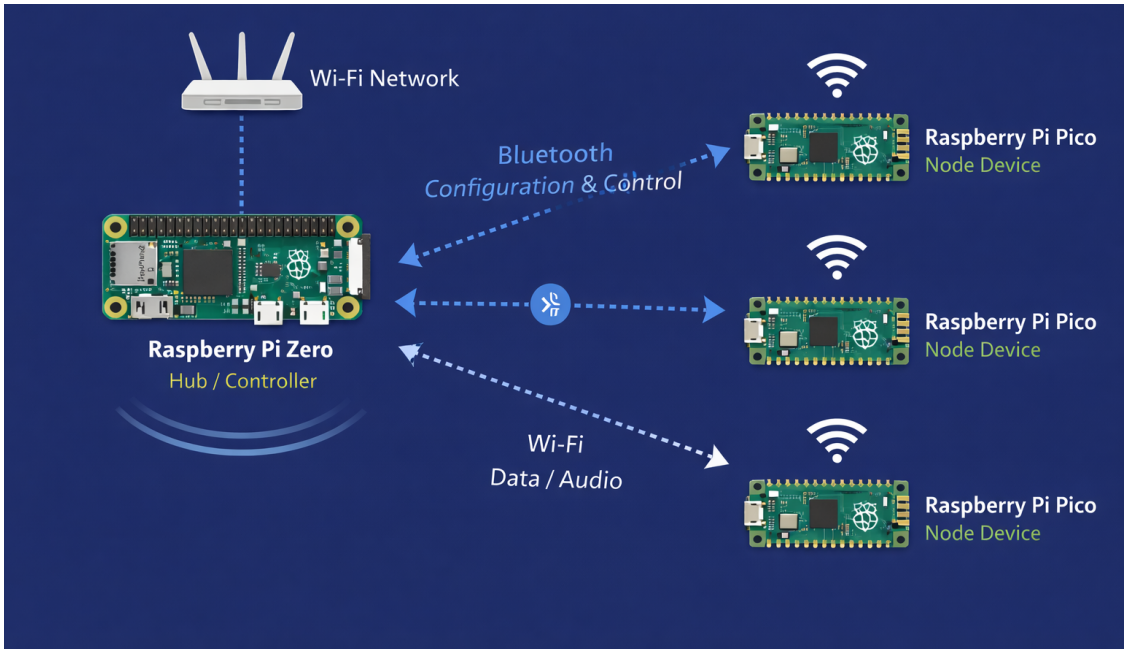
POTATAO

Group Portfolio - Period 3



Group members

Darhan Kumysbekov

Francisco Pimentel

Maksym Hrapyniuk

Sunny Wang

Vladyslav Pernarovskyi

Table of Contents

Introduction *3*

Technical Design *4*

Design Decisions **4**

System Architecture **6**

Hardware Pico Layer 6

Hardware Zero Layer 6

Network Communication Layer 7

Component Information - Test Report | Researches **8**

Raspberry PI Pico 8

Raspberry PI Zero (Base Station) 11

Requirements **13**

User Stories **14**

Secure voice storage 14

Favorite chats 14

Receive untranslated messages 14

Language and translation settings 14

Device design 14

Limitations **15**

Future Improvement **15**

Web application 15

Security Layer 16

Test Scripts *17*

Introduction

Potatao is a student project developed by a group of five students at NHL Stenden University of Applied Sciences during Period 3 of 2026. The project aims to design and build a compact, portable communication device capable of recording, transmitting, and playing back audio messages across language barriers, in real time.

At its core, Potatao is a hardware-software system built around the Raspberry Pi Pico 2W and Raspberry Pi Zero platforms. The portable "Potatao phone" allows users to record voice messages in their native language, which are then wirelessly transmitted to a base station. The base station, powered by an AI translation server (Fast-Whisper), transcribes and translates the audio into the listener's preferred language before delivering it back to the target device. This makes Potatao not just a communication tool, but a real-time cross-language audio messaging system.

The project integrates a range of technologies, including I2S audio interfaces, NRF24L01 radio modules, OLED displays, and a web application backend built on PostgreSQL and Redis, into a single, cohesive prototype. The software stack combines C++ for low-level microcontroller logic with Python for server-side communication and data processing.

This portfolio documents the design decisions, system architecture, hardware components, and requirements that shaped the development of Potatao, as well as the team's vision for future improvements.

Technical Design

Design Decisions

During our meeting, we discussed many details of our project. As a concept, it has already gone through many changes. At the beginning, each team member had a variety of ideas, but as the discussion continued, many of them were changed, combined, or set aside. It was important for a project like ours to set strict and achievable goals in order to get the most polished result possible. We wanted the device to be well-made, reliable, and carefully designed. After making several decisions, our final project design looks like this.

The device we designed will be a compact box that contains the following components:

- Raspberry Pi Pico 2W
- Radio module
- Battery supply
- OLED screen
- Potentiometer
- Buttons
- SD card module with SD card
- Microphone
- Speaker (dynamic speaker)

The radio module will be used to transfer data between a small server (in our case we use Raspberry Pi Zero) and devices. The battery supply will power the device, making it portable. The OLED screen will display the user interface and important information. Buttons and the potentiometer will be used to control the device and navigate through the interface. The SD card module and SD card will be used to store data that the device receives or sends. The microphone will be used to record audio data, and the speaker will be used to play audio on the receiving side.

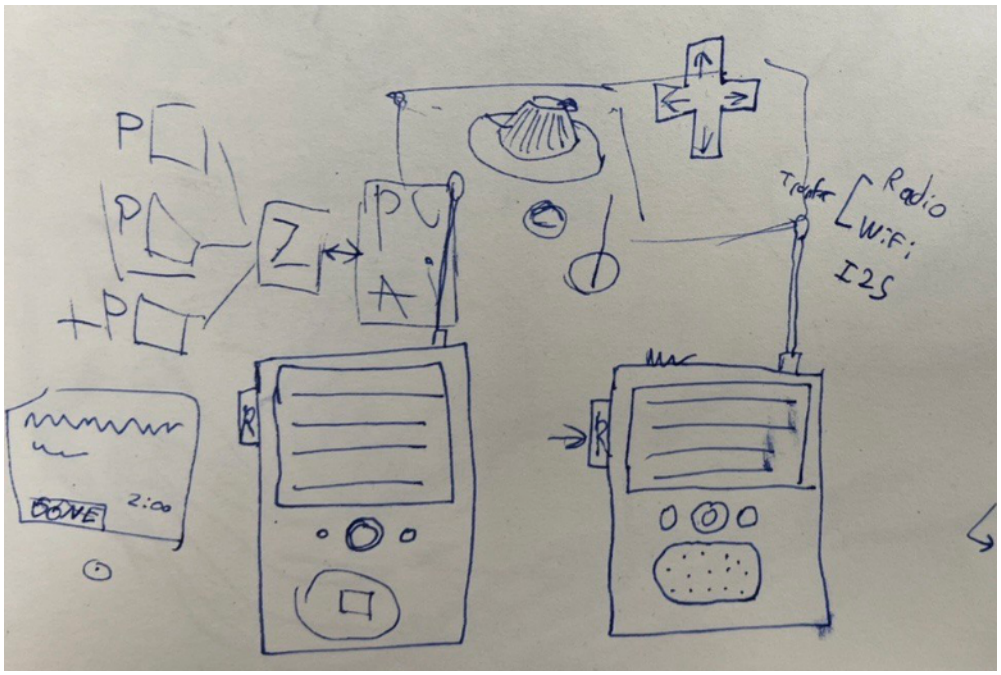
In addition to the main device, there will also be a station built on a Raspberry Pi Zero. The station consists of:

- Raspberry Pi Zero
- Radio module

The station will be used to receive, analyze, and transform the data sent from the main device. It will also be able to send audio data back to the device. Essentially, the station acts as a base unit that processes information and communicates with the portable device.

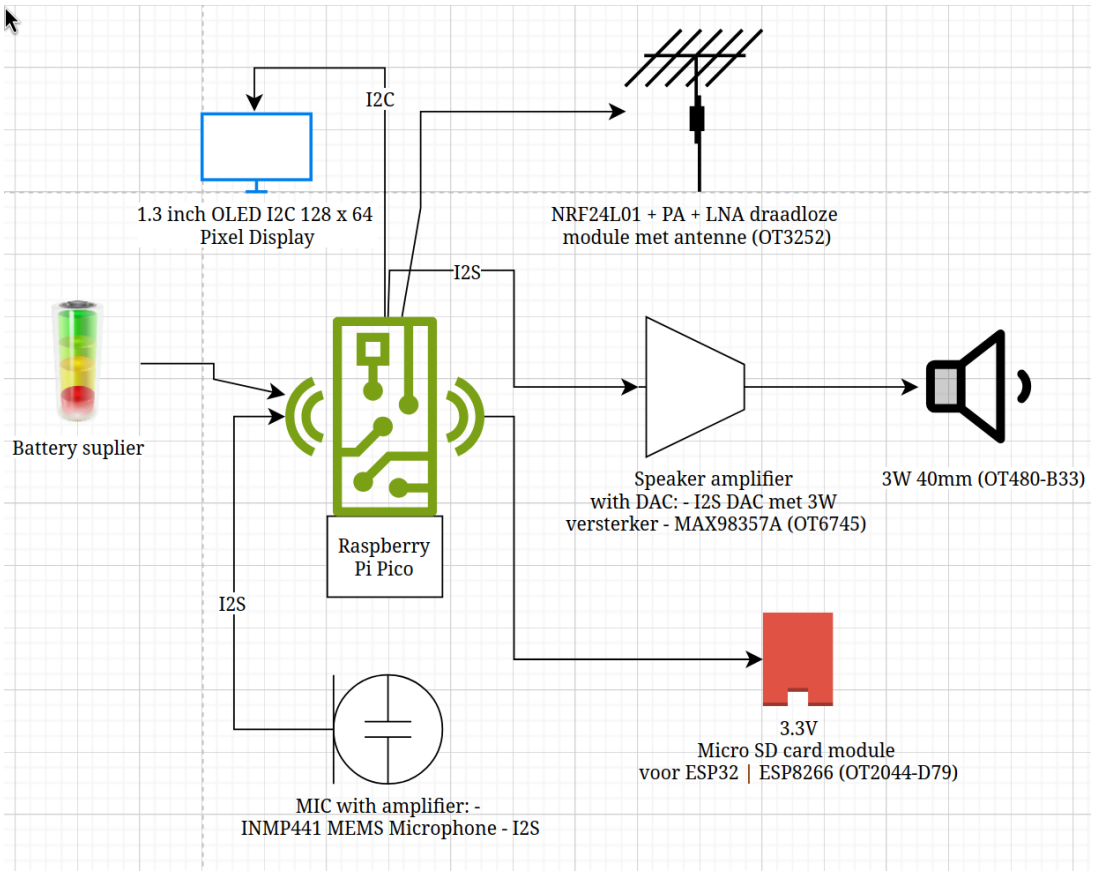
The main programming language for the device will be C++, because it is fast and reliable enough for our goals and works well with microcontrollers. We will also use Python for sending requests, handling communication with the station, and possibly for data processing on the Raspberry Pi Zero.

Overall, our goal is to create a compact and reliable communication device that can send and receive data, store information, and transmit audio. The project focuses on combining hardware and software into one complete system. We aim to make the device simple to use, but technically well-designed inside. If everything works as planned, the final result should be a fully functional prototype that demonstrates wireless data and audio communication between two devices.

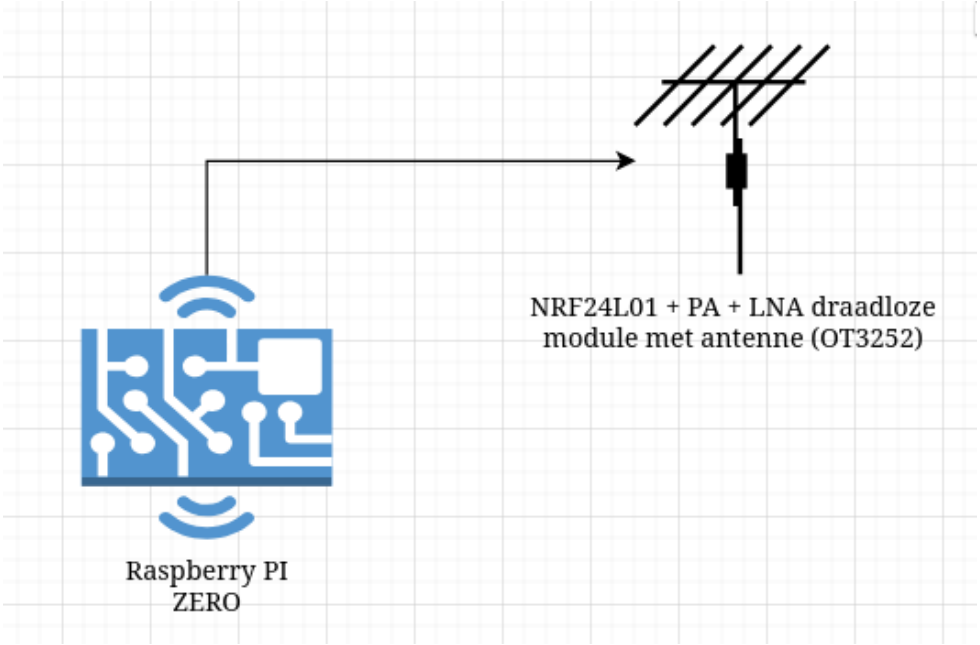


System Architecture

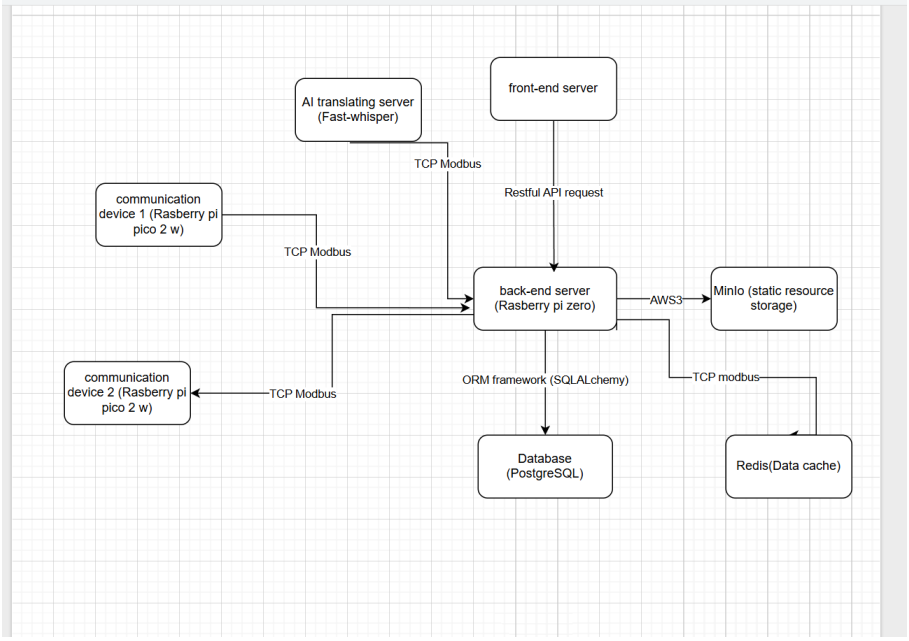
Hardware Pico Layer



Hardware Zero Layer



Network Communication Layer



For this project, Raspberry Pi Pico 2W is used to send user original language voice information and receive other Pico device voice information that they choose the ideal language. Before communication starts, users should choose the language they listen to and also choose one of the channels (rooms). When users finish voice message input, this voice message should be transmitted to Raspberry Pico Zero, which is back-end server. Zero transfers this data to AI translating service. AI translating service translates this voice message to requested languages and sends it back to Zero. Those translated voice messages are sent to the targeted Pico 2W devices.

Component Information - Test Report | Researches

Raspberry PI Pico

The Raspberry Pi Pico serves as the core processing unit of the device. As a microcontroller-based platform, it is responsible for handling all low-level hardware interactions, including communication with peripherals, audio data acquisition, and control of the user interface.

Unlike single-board computers, the Pico operates without a full operating system, which allows for highly predictable and efficient execution. This makes it particularly suitable for real-time tasks such as audio streaming, signal processing, and device control.

However, due to its hardware limitations, the Pico cannot directly drive components that require higher power, such as speakers, or process very low-level analog signals from microphones. For this reason, additional components such as amplifiers and digital interfaces are required.

Speaker amplifier (DAC + I2S) - MAX98357A | 3W I2S DAC amplifier

The speaker amplifier is a critical component that enables audio playback from the Pico. Since a microcontroller cannot provide sufficient current to drive a speaker directly, an external amplifier is required.

The selected module (MAX98357A) integrates both a Digital-to-Analog Converter (DAC) and a Class-D amplifier. This allows it to convert digital audio signals received via the I2S protocol into analog signals and amplify them to a level suitable for driving a speaker.

The use of I2S ensures efficient and high-quality audio transmission with minimal CPU overhead, making it ideal for embedded audio applications.

Speaker - 3W 40mm | OT480-B33

The speaker is responsible for outputting the audio signal to the user. A 3W, 40mm speaker provides a good balance between size, power consumption, and sound quality for a portable embedded device.

The system is designed using a half-duplex communication model, meaning the device either records or plays audio at a given time, but not both simultaneously. This simplifies the hardware and software design, reduces processing requirements, and avoids audio feedback issues.

Storage - Micro SD module ESP32/ESP8266 | OT2044-D79

The storage system is based on a 3.3V, compatible Micro SD card module, which is fully compatible with the Raspberry Pi Pico's voltage levels.

The SD card is used to store locally:

- audio data
- message logs
- configuration data

The card must be formatted using the FAT filesystem and is recommended to have a capacity of at least 32GB.

For reliable data handling, the system is implemented in C/C++ rather than MicroPython. This is because SD card operations require precise timing and higher performance, and interpreted environments may introduce latency or data loss during intensive read/write operations.

Mic (with amplifier, I2S) - INMP441 MEMS Microphone

The INMP441 is a digital MEMS microphone with an integrated amplifier and I2S interface, making it well-suited for embedded audio applications.

Traditional microphones produce very low-level analog signals, which require external amplification and analog-to-digital conversion. In contrast, the INMP441 performs both amplification and digitization internally, outputting a clean digital audio stream directly over the I2S interface.

This significantly simplifies the hardware design and improves signal quality, as it reduces noise and eliminates the need for additional analog circuitry.

The use of I2S also ensures efficient and synchronized audio data transfer between the microphone and the microcontroller, which is essential for stable audio recording.

Display - 1.3 inch OLED I2C 128 x 64 Pixel Display

This display serves as the primary visual interface of the Potatao device. With a resolution of 128 by 64 pixels, it is capable of rendering clear text and simple graphical elements within a compact form factor, making it well suited for embedded systems where space is limited.

Communication between the display and the Raspberry Pi Pico is established via the I2C protocol, which requires only two data lines, SDA and SCL, reducing wiring complexity and preserving GPIO pins for other peripherals. This is particularly relevant in the Potatao hardware setup, where multiple components share the same microcontroller.

In the context of this project, the display is responsible for presenting the graphical user interface, including navigation menus, status indicators, and feedback during audio recording and playback operations.

Battery - 2500 mAh lithium batteries

The Potatao device is designed to operate on a portable battery supply, ensuring full independence from fixed power sources. The recommended configuration consists of two 2500 mAh lithium batteries managed by a Battery Management System (BMS), which provides essential protection functions such as overcharge, over-discharge, and short-circuit prevention.

The power architecture is built around a common 5V bus, as the primary components of the device, including the Raspberry Pi Pico, the OLED display, and the NRF24L01 radio module, all operate at

this voltage level. In cases where a higher voltage is required, such as for the audio amplifier, a boost converter can be integrated into the circuit to step up the voltage accordingly.

As an alternative, a standard power bank may also be used as the power source, offering a simpler and more readily available solution, particularly during the prototyping and testing phases of the project.

Radio - NRF24L01 + module sans fil PA + LNA avec antenne | OT3252

The NRF24L01 is a wireless communication module that operates at a frequency of 2.4 GHz, the same frequency band used by standard Wi-Fi networks. When equipped with an external antenna, the module is capable of achieving a transmission range of approximately 500 to 700 metres, making it a suitable solution for short to medium-range wireless communication applications.

From a regulatory standpoint, the use of this module presents a significant advantage over conventional AM/FM radio transmission methods. Broadcasting live radio signals on AM/FM frequencies requires the operator to hold a valid broadcast license. The NRF24L01, however, operates on the 2.4 GHz ISM band, which is freely available for use without the need for such licensing, making it a legally accessible option for voice transmission projects.

Raspberry PI Zero (Base Station)

The Raspberry Pi Zero functions as the central base station and processing unit of the system. Unlike the Raspberry Pi Pico, which operates as a microcontroller, the Zero runs a full Linux-based operating system, allowing it to perform more complex tasks such as data processing, storage management, and communication coordination.

In this project, the Raspberry Pi Zero acts as a **communication server and routing node**. It continuously listens for incoming data from connected devices (Potatao Phones), processes the received information, and forwards it to the appropriate recipients based on the selected communication channel.

Its responsibilities include:

- Receiving audio and control data from devices
- Managing connected device IDs and communication channels
- Routing messages to one or multiple receivers

- Acting as a bridge between different communication layers (radio and higher-level services)
- Optionally processing data (e.g., speech-to-text, translation, text-to-speech)

Thanks to its higher computational capabilities compared to the Pico, the Zero is also suitable for running additional services such as databases, APIs, and data processing pipelines.

From a hardware perspective, the Raspberry Pi Zero requires minimal additional components. Its primary external interface in this project is the radio module, which enables wireless communication with the portable devices.

Radio Module (NRF24L01 + PA + LNA) | *OT3252*

The NRF24L01 radio module is used as the primary wireless communication interface between the Raspberry Pi Zero (base station) and the Raspberry Pi Pico devices.

Operating on the 2.4 GHz ISM band, the module allows bidirectional data transmission without requiring a licensed frequency. When equipped with a Power Amplifier (PA) and Low Noise Amplifier (LNA), along with an external antenna, it can achieve communication ranges of approximately 500 to 700 metres under optimal conditions.

In the context of the base station, the radio module serves as:

- A receiver for incoming data from multiple devices
- A transmitter for sending processed or relayed data back to selected devices

This enables the Raspberry Pi Zero to act as a centralized communication hub, coordinating data exchange between all connected nodes in the system.

The use of the NRF24L01 provides a lightweight and efficient alternative to Wi-Fi, with lower power consumption and simpler protocol control, which is particularly beneficial for embedded communication systems.

Requirements

Must Have – Portable Device (“Potatao Phone”)

The portable device must:

- Be able to connect to the Raspberry Pi Zero base station
- Send and receive data via the radio module
- Record audio using the microphone
- Play audio through the speaker
- Provide a graphical user interface (GUI) using the OLED screen
- Allow user interaction through buttons and a potentiometer
- Store data locally using an SD card (e.g., messages or logs)
- Allow the user to select a preferred language

Must Have – Base Station (Raspberry Pi Zero)

The base station must:

- Receive data from the portable devices
- Process incoming audio data
- Convert speech to text (Speech-to-Text)
- Translate the received text into another language
- Convert translated text back into audio (Text-to-Speech)
- Send processed audio/data back to devices
- Store device IDs and manage connected devices
- Maintain a database of text messages and communication logs

Could Have (Optional Features)

- Data encryption between devices and base station
- Chat rooms / communication channels
- Conversation titles or message grouping
- Confirmation/approval before sending audio (configurable in settings)
- Selection of preferred voice type (e.g., male/female voice)
- Ability to subscribe to specific channels or rooms
- Web-based frontend to view data and manage communication

- Ability to create and manage rooms/channels (via device or web interface)

Advanced / Future Features (“Crazy to Have”)

- AI-powered communication channel with a private assistant
- Ability to integrate custom AI models
- Users can join AI channels and receive responses on their device
- Direct user-to-user audio messaging system

User Stories

Secure voice storage

“As a user, I want the ability to store my voice messages securely on my device so that my data remains private.”

Favorite chats

“As a user, I want to be able to add chats or friends to a favorites list so that I can access them quickly.”

Receive untranslated messages

“As a user, I want to receive voice messages without translation when I understand the language so that I can hear the original content without AI voice processing to hear original voice, because it sounds more natural for me.”

Language and translation settings

“As a user, I want to configure my preferred language for translations and select languages that should not be translated, as well as I have access to transcriptions in the original language so that I can customize my communication experience, due to the fact that I know a lot of languages and I pretend to learn more.”

Device design

“As a user, I want the device to be compact and well-designed so that I can easily use it with one hand.”

Limitations

Since the project is currently still in the design phase, the limitations identified below are based on technical research and anticipated challenges rather than live testing results. However, these are considered significant factors that could affect the system's performance once implemented.

The limitation of Pico. It doesnt support TLS and has a small memory size to store data and send data.

One of the primary concerns is the limited range of the radio and Wi-Fi communication. The NRF24L01 module used in the design has a relatively short effective range, particularly in environments with physical obstacles or interference. This could restrict the practical use of the Potatao phone to small indoor spaces, which limits its versatility as a portable communication device.

Audio recording quality is another anticipated limitation. The INMP441 MEMS microphone, while compact and suitable for the hardware setup, may struggle to capture clear audio in noisy environments. Background noise could negatively impact the accuracy of the AI transcription and translation pipeline, leading to errors in the final delivered message.

Future Improvement

Web application

Web application is a web platform which users can see chatting channels tha they join and create. Users can create or modify channels. Users can see the whole conversation on every chatting channel. The conversation is based on the audio message; user can listen to these audio messages by targeted language.

Additionally, the team would like to develop a more complete frontend web application. The current design includes a basic web platform for viewing chat channels, but future iterations could include a fully featured interface with user authentication, message history, channel management, and potentially a mobile-friendly or native app version. This would make Potatao accessible beyond the physical hardware device, broadening its potential user base.

Security Layer

As a future improvement, the system can be enhanced by introducing a dedicated secure network layer based on the Raspberry Pi Zero.

The idea is to configure the Raspberry Pi Zero to operate simultaneously as:

- a **client device**, connected to the external internet via Wi-Fi
- a **local access point (hotspot)** for all Potato Phone devices

This can be achieved by using an additional external Wi-Fi module, allowing the Zero to manage two network interfaces at the same time.

In this architecture, all Raspberry Pi Pico devices would connect only to the local network created by the Zero, rather than directly to external networks. This introduces a clear **security boundary**, where the Raspberry Pi Zero acts as the only gateway between internal devices and the outside world.

The benefits of this approach include:

- **Network isolation** – Pico devices are not directly exposed to external networks
- **Controlled access** – only devices connected through the Zero can communicate within the system
- **Improved security** – all incoming and outgoing traffic can be monitored, filtered, or restricted at the Zero level
- **Reduced attack surface** – unauthorized users cannot access Pico devices without first gaining access to the Zero

This architecture effectively creates a private and controlled communication environment, which is especially valuable for embedded and IoT systems where security and reliability are critical.

Test Scripts

The test report shows complete test results that feature sensors and actuators of the Innovative system related to the available manufacturer data, using appropriate equipment and/or tools.

The causes of possible failures have been fully investigated and appropriate conclusions drawn. The test results are reproducible.

Test ID	Description	Priority	Status
TC-01	Basic communication between two Raspberry Pico 2 W	High	
TC-02	The setting up of Raspberry Pi Zero	High	
TC-03	Data storage for Raspberry Pi Pico 2 W in SD card.	High	
TC-04	Basic communication between Raspberry Pi Pico 2 W and Raspberry Pi Zero.	High	
TC-05	Local database installation in Raspberry Pico 2 W	High	
TC-06	Play audio with speaker	High	
TC-07	Speak with microphone.	High	
TC-08	Provide a graphical user interface (GUI) using the OLED screen	High	
TC-09	Test APIs related to chatting channels	Medium	

The test scripts and test cases are comprehensive and exceptionally well-documented. All key aspects, what is being tested, test steps, expected results, actual results, and test status, are clearly and thoroughly described. The test cases cover all critical functionalities, edge cases, and potential failure scenarios. The documentation is structured, easy to follow, and demonstrates a professional level of quality.

Detailed Test Cases

Test Case ID: TC-01

Name: Basical communication between two Raspberry Pi Pico 2 W

Objective: To test that Raspberry Pi Pico 2 W can transmit data to each other.

Preconditions:

- Raspberry Pi Pico 2 W can transmit normal data which format is Json via WIFI and Radio.
- Raspberry Pico 2 W can transmit audio data by streaming via WIFI and Radio.

Test steps for normal data:

- Write a testing Micropython script
- Put this script into Raspberry Pi Pico 2 W
- Send normal Json data to another Raspberry Pi Pico 2 W with microdot by WIFI and another protocol by Radio.

Test steps for audio data:

- Use speaker to say some words
- Another Raspberry Pi Pico 2 W can receive this audio and play immediately.

Expected Result: Raspberry Pi Pico 2 W can send and receive data.

Actual Result: Raspberry Pi Pico 2 W can send and receive data successfully

Test Status:

Test Case ID: TC-02

Name: The setting up of Raspberry Pi Zero

Objective: To verify whether remote server in Raspberry Pi Zero can be accessed successfully

Preconditions:

- Can access remote Linux server in Raspberry Pi Zero successfully

Test steps:

- Download Xshell (remote server management software)
- Fill the IP and password about remote server
- Check whether remote server can be visualized in Xshell

Expected Result: Users can see the Terminal with remote server in Xshell.

Actual Result: Users can successfully see the Terminal with remote server in Xshell.

Test Status:

Test Case ID: TC-03

Name: Data storage for Raspberry Pi Pico 2 W in SD Card

Objective: To verify whether data can be stored in SD card successfully

Preconditions:

- Can store the data in SD card successfully

Test steps:

- Write a testing C++ script for connecting SD card data storage function
- Put this testing script into Raspberry Pi Pico 2 W
- Check whether this script can get SD card information successfully
- Check whether this script can generate Json or txt file in SD card
- Check whether this script can write some words in Json or txt file

Expected Result: User can see the SD card information and files and context in files in SD card

Actual Result: User can see the SD card information and files and context in files in SD card successfully

Test Status:

Test Case ID: TC-04

Name: The basic communication between Raspberry Pi Pico 2 W and Raspberry Pi Zero

Objective: To verify whether Raspberry Pi Zero and Raspberry Pi Pico 2 W can transfer data with each other via WIFI and Radio

Preconditions:

- Raspberry Pi Zero and Raspberry Pi Pico 2 W can send and receive the data successfully

Test steps - Communication with WIFI:

- Initialize Wi-Fi on Pico 2 W and Pi Zero; connect to the same SSID.
- Start a TCP Server script on Pi Zero listening on Port 8080.
- Execute TCP Client on Pico 2 W to connect to Pi Zero's IP.
- Send a JSON string (e.g., {"sensor": "temp", "val": 25.5}) from Pico 2 W.
- Send an ACK (Acknowledgment) message from Pi Zero back to Pico 2 W.

Test steps - Communication with Radio:

- Initialize Radio modules on both devices using SPI.
- Set both devices to the same Frequency/Channel and Address.
- Set Pico 2 W to TX (Transmitter) and Pi Zero to RX (Receiver).
- Pico 2 W sends a 32-byte payload containing a timestamp.

Expected Result:

- User can see the Json data in Pico 2 W or Pi Zero
- User can see the 32-bytes payload data in Pico 2 W and Pi Zero

Actual Result:

- User can see the Json data in Pico 2 W or Pi Zero successfully
- User can see the 32-bytes payload data in Pico 2 W and Pi Zero successfully

Test Status:

Test Case ID: TC-05

Name: Local database installation in Raspberry Pico 2 W

Objective: To verify whether local database (SQLite) can be installed in Raspberry Pico 2 W successfully

Preconditions:

- Can access local database (SQLite) in Raspberry Pico 2 W successfully

Test steps:

- Download Navicat (Database management software)
- Fill the Pico IP and database port, username, password, DB name.
- Check whether database can be visualized in Navicat

Expected Result: User can access database by Navicat

Actual Result: User can access database by Navicat successfully

Test Status:

Test Case ID: TC-06

Name: Play audio with speaker

Objective: To verify that speaker can play audio file from Pico 2 W SD card or from Pi Zero.

Preconditions:

- Speakers can play audio files from Pico 2 W SD card or from Pi Zero.

Test steps:

- Put a testing audio file into Pico 2 W and Pi zero
- Write a testing C++ script for reading and playing audio files
- Check whether the speaker can play audio file.

Expected Result: User can listen to the targeted audio file from the speaker

Actual Result: User can successfully listen to the targeted audio file from the speaker

Test Status:

Test Case ID: TC-07

Name: Speak with microphone.

Objective: To verify that Pico 2 W can receive the User speech from microphone to convert into audio and store audio file into local SD card or Pi Zero.

Preconditions:

- Pico 2 W can receive the user speech and convert to audio file.
- Pico 2 W can store this audio file into local SD card or Pi Zero.

Test steps:

- Write a testing C++ script for receiving the user speech and storing the audio file in local SD card or Pi Zero.
- User speaks some words with microphone
- Check this audio file whether it exists in local SD card or Pi Zero

Expected Result: Audio file can be stored in local SD card or Pi Zero.

Actual Result: Audio file can be stored in local SD card or Pi Zero successfully

Test Status:

Test Case ID: TC-08

Name: Provide a graphical user interface (GUI) using the OLED screen

Objective: To verify that users can see chatting channels that they created or joined on the OLED screen.

Preconditions:

- Users can see the chatting channels information that they created or joined

Test steps:

- Start the Pico 2 W
- Check the chatting channels information on the OLED screen

Expected Result: Users can see the chatting channels information that they created or joined

Actual Result: Users can see the chatting channels information that they created or joined successfully

Test Status:

Test Case ID: TC-09

Name: Test APIs related to chatting channels

Objective: To verify whether APIs which are related to chatting channels can be accessed.

Preconditions:

- APIs can be accessed by Postman

Test steps:

- Open Postman (API test software)
- Fill the targeted API information in the Postman
- Check the API response result

Expected Result: APIs provider the correct response information in Postman

Actual Result: APIs provider the correct response information in Postman successfully

Test Status:

Links

Linear

Link: https://linear.app/potatao/project/the-defpic-a647738d6dcd/overview

GitHub

Link: https://github.com/Fumbre/potatao